SpectraLens: Weight Spectra, Loss Curvature and Low-Rank Structure of Small Vision Networks

Huy Pham

November 2026

Abstract

We study three small image classifiers (a VGG-style CNN, ResNet-18 and a tiny ViT) trained from scratch on EuroSAT and CIFAR-10 through the lens of numerical linear algebra. All matrix algorithms (power iteration, Lanczos with full reorthogonalization, randomized SVD, Hutchinson and Hutch++ trace estimation, stochastic Lanczos quadrature, Hessian-vector products) are implemented from scratch and verified against reference solvers. We find that (i) at initialization every layer matches the Marchenko–Pastur prediction for random matrices, while training halves the stable rank of convolutional layers, pushes 5–15% of the eigenvalues out of the noise bulk and makes the spectral tails heavy (Hill exponent 9–13 $\rightarrow$ 3–4.6); (ii) growing the batch size from 32 to 512 makes the final minimum 5.4$\times$ sharper (top Hessian eigenvalue 44.5 $\rightarrow$ 240.7) and costs 1.2 points of test accuracy, while SAM lowers the top eigenvalue by 44% without improving accuracy on this dataset; (iii) Tucker-2 compression keeps the CNN at full accuracy with 45% of the multiply-accumulates after a two-epoch fine-tune and ResNet-18 within 0.33 points at 59%, whereas the ViT is close to full rank and barely compresses; and (iv) a layer's stable rank only weakly predicts how much accuracy is lost when that layer alone is truncated (Spearman $\rho = 0.31$ on ResNet-18).

1 Introduction

A trained network is, to a large extent, a collection of matrices. Their spectra say how much of each layer is structure and how much is indistinguishable from noise; the spectrum of the loss Hessian says how sharp the minimum is that the optimizer found; and the singular-value decay of a layer bounds how well it can be replaced by a cheaper low-rank factorization. This report asks three questions on deliberately small models, where every quantity can be computed and checked:

1. How do weight spectra evolve during training, and how far do they move from a random (Marchenko–Pastur) baseline?
2. How do the top Hessian eigenvalues and the Hessian trace depend on the batch size, and what does sharpness-aware minimization (SAM) change?
3. How much accuracy does low-rank compression cost per multiply-accumulate saved, and does the spectrum predict which layers are safe to compress?

Section 2 describes the linear algebra and the guarantees it relies on, Section 3 the models and data, and Sections 4–6 the three experiments. All numbers are reproducible with the commands in Appendix A.

2 Numerical toolkit

Every routine accepts either a dense matrix or a matrix-free operator $v \mapsto Av$, so the same code runs on a 4096×4096 test matrix and on the Hessian of a network with 10^7 parameters.

2.1 Best low-rank approximation

Theorem 1 (Eckart–Young–Mirsky [1])**.** *Let $A = U\Sigma V^\top$ with singular values $\sigma_1 \ge \sigma_2 \ge \dots$ and let $A_k = U_k \Sigma_k V_k^\top$ keep the top k triplets. For every B of rank at most k,*

$$\|A - B\|_2 \ge \|A - A_k\|_2 = \sigma_{k+1}, \qquad \|A - B\|_F \ge \|A - A_k\|_F = \Big(\sum_{j>k} \sigma_j^2\Big)^{1/2}.$$

Proof sketch (spectral norm). If $\operatorname{rank}(B) \le k$, its null space has dimension at least $n - k$, so it intersects the $(k+1)$-dimensional span of $v_1, \dots, v_{k+1}$ in a unit vector w. Then $\|(A - B)w\| = \|Aw\| \ge \sigma_{k+1}$, because w lies in a subspace on which A stretches every vector by at least σ_{k+1}. □

Truncated SVD is therefore optimal per layer; the compression experiments test how far "optimal per layer" is from "harmless for the network". Our test suite checks that the rank-r factorization of a random layer has spectral error equal to σ_{r+1} to 10^{-5} relative precision.

2.2 Randomized SVD

The randomized range finder of Halko, Martinsson and Tropp [4] samples $Y = (AA^\top)^q A\Omega$ with a Gaussian $\Omega \in \mathbb{R}^{n \times (k+p)}$, orthonormalizes $Y = QR$ and computes the SVD of the small matrix $Q^\top A$.

Theorem 2 ([4], Thm. 10.6 and Cor. 10.10)**.** *With oversampling $p \ge 2$ and no power iterations,*

$$\mathbb{E}\|A - QQ^\top A\|_2 \le \Big(1 + \sqrt{\tfrac{k}{p-1}}\Big)\sigma_{k+1} + \frac{e\sqrt{k+p}}{p}\Big(\sum_{j>k} \sigma_j^2\Big)^{1/2},$$

and q power iterations replace σ_j by σ_j^{2q+1} inside the bound, then take the $(2q+1)$-th root, so the excess over σ_{k+1} decays geometrically in q.

The bound is checked against Hypothesis-generated matrices in the test suite. On a 4096×4096 matrix with $\sigma_i = 1/i$ (Table 1), two power iterations bring the rank-64 approximation within 2% of the optimal error σ_{65} at $650\times$ less time than a full SVD.

2.3 Lanczos and its convergence

m steps of Lanczos build an orthonormal Krylov basis Q_m with $AQ_m = Q_m T_m + \beta_m q_{m+1} e_m^\top$, T_m tridiagonal; the eigenvalues of T_m (Ritz values) approximate the extreme eigenvalues of A. In floating point the basis loses orthogonality once a Ritz value converges and spurious copies ("ghosts") appear; we reorthogonalize every new vector against the whole basis twice ("twice is enough"), at $O(nm^2)$ extra work. Breakdown is declared when $\beta_j < \sqrt{\varepsilon}\,\|A\|$, a tolerance chosen so that numerically rank-deficient test matrices (null eigenvalues $\sim 10^{-16}$) still stop cleanly.

Method (4096×4096, float32, laptop GPU)	Time (ms)	Error
full SVD (`torch.linalg.svd`)	5633.6	1.00
randomized SVD, rank 64, $q = 0$	5.5	2.67
randomized SVD, rank 64, $q = 2$	8.6	1.02
dense `eigvalsh`	248.0	4.8×10^{-6}
Lanczos, top 10, 60 steps	28.4	3.8×10^{-6}

Table 1: Toolkit benchmark. SVD error is the spectral residual divided by the optimum σ_{65} on the spectrum $\sigma_i = 1/i$; eigenvalue error is the largest absolute error of the top 10.

Theorem 3 (Kaniel–Paige–Saad [15])**.** *Let $\lambda_1 > \lambda_2 \geq \cdots \geq \lambda_n$, ϕ the angle between the start vector and the top eigenvector, and $\gamma = (\lambda_1 - \lambda_2)/(\lambda_2 - \lambda_n)$. The largest Ritz value after m steps satisfies*

$$0 \leq \lambda_1 - \theta_1^{(m)} \leq (\lambda_1 - \lambda_n) \left(\frac{\tan \phi}{T_{m-1}(1 + 2\gamma)} \right)^2,$$

where T_{m-1} is the Chebyshev polynomial of degree $m - 1$.

Because $T_{m-1}(1 + 2\gamma)$ grows like $e^{2(m-1)\sqrt{\gamma}}$, Lanczos needs roughly $1/\sqrt{\gamma}$ steps where power iteration needs $1/\gamma$; on our test matrix, 60 steps give the top 10 eigenvalues to 3.8×10^{-6} in 28 ms versus 248 ms for a dense solver.

2.4 Trace estimation and spectral density

For a symmetric A and probes z with $\mathbb{E}[zz^\top] = I$, Hutchinson's estimator [6] $\operatorname{tr}(A) \approx \frac{1}{s} \sum_i z_i^\top A z_i$ is unbiased with single-probe variance $2(\|A\|_F^2 - \sum_i A_{ii}^2)$ for Rademacher probes. Hutch++ [13] first computes the trace of A restricted to an approximate top eigenspace exactly and applies Hutchinson only to the deflated remainder, needing $O(1/\varepsilon)$ instead of $O(1/\varepsilon^2)$ products for relative error ε on PSD matrices.

The full eigenvalue *density* $\phi(t) = \frac{1}{n} \sum_j \delta(t - \lambda_j)$ comes from stochastic Lanczos quadrature (SLQ) [10, 3]. For a unit probe v, m Lanczos steps give $T_m = S \operatorname{diag}(\theta) S^\top$, and

$$v^\top f(A)\, v \;\approx\; \sum_{i=1}^{m} \tau_i \, f(\theta_i), \qquad \tau_i = S_{1i}^2,$$

which is exactly the m-point Gauss quadrature rule for the spectral measure of v and is exact for polynomials of degree up to $2m - 1$. Averaging over random probes ($\mathbb{E}[vv^\top] = I/n$) and smoothing each node with a Gaussian kernel gives the density plots in Figure 2; $f(t) = t$ recovers the Hutchinson trace estimate.

2.5 Hessian-vector products

For a loss $L(\theta)$ with gradient g, Pearlmutter's identity [14] $Hv = \nabla_\theta(g^\top v)$ costs about two backward passes and never forms the $P \times P$ Hessian (for ResNet-18, $P \approx 1.1 \times 10^7$ and the dense Hessian would need about 500 TB in float32). Our batched operator recomputes the double-backward graph batch by batch on every product, weighting by batch size, so memory stays at one batch; a test checks it equals the Hessian of the concatenated batch exactly.

Model	Params	EuroSAT	CIFAR-10
Small CNN (3 VGG blocks)	0.29 M	97.78%	88.64%
ResNet-18 (3×3 stem, no max-pool)	11.2 M	**97.96%**	**92.30%**
Tiny ViT (6 blocks, width 192, 64 tokens)	1.8 M	93.56%	78.08%

Table 2: Test accuracy after 30 epochs, one seed per run.

Run	Stable-rank ratio	Beyond MP edge	α (median)
CNN, EuroSAT	$0.53 \to 0.19$	$0.0\% \to 15.0\%$	$9.3 \to 3.1$
CNN, CIFAR-10	$0.53 \to 0.28$	$0.0\% \to 12.2\%$	$9.3 \to 3.9$
ResNet-18, EuroSAT	$0.53 \to 0.27$	$0.1\% \to 7.6\%$	$13.2 \to 4.6$
ResNet-18, CIFAR-10	$0.53 \to 0.17$	$0.1\% \to 12.5\%$	$13.2 \to 3.4$
ViT, EuroSAT	$0.30 \to 0.20$	$0.3\% \to 2.4\%$	$6.3 \to 5.2$
ViT, CIFAR-10	$0.31 \to 0.13$	$0.0\% \to 4.8\%$	$6.5 \to 3.1$

Table 3: Weight spectra at initialization $\to$ after 30 epochs (medians over layers).

3 Models and data

EuroSAT RGB [5] (27,000 Sentinel-2 tiles, 64×64, 10 land-use classes) is split 80/10/10 stratified; CIFAR-10 [9] uses its official test set and a 90/10 train/validation split. Normalization statistics come from the training split only. Three architectures are trained from scratch for 30 epochs (SGD with Nesterov momentum 0.9, learning rate 0.1, one warm-up epoch then cosine decay, weight decay 5×10^{-4} except on biases and normalization layers, bf16 autocast), keeping checkpoints at epochs $0, 1, 2, 5, 10, 20, 30$ (Table 2).

4 Weight spectra

Each weight is viewed as a matrix ($W \in \mathbb{R}^{c_{\text{out}} \times c_{\text{in}} k^2}$ for convolutions; attention's packed projection is split into W_Q, W_K, W_V), and three quantities are tracked:

Definition 1. The *stable rank* is $\operatorname{srank}(W) = \|W\|_F^2 / \|W\|_2^2 \in [1, \operatorname{rank} W]$; we report it as a fraction of $\min(m, n)$. The *outlier fraction* is the share of eigenvalues of $WW^\top/n$ above the Marchenko–Pastur edge $\lambda_+ = \sigma^2(1 + \sqrt{m/n})^2$ ($m \le n$), where $\sigma^2 = \|W\|_F^2/(mn)$ is the entry variance. The *tail exponent* α is the Hill estimate $\hat{\alpha} = 1 + k \,/ \sum_{i=1}^{k} \log(\lambda_i / \lambda_{k+1})$ on the largest 10% of the eigenvalues.

The Marchenko–Pastur law [11] says that for a matrix of i.i.d. entries with variance σ^2 the eigenvalues of $WW^\top/n$ fill $[\sigma^2(1-\sqrt{q})^2, \sigma^2(1+\sqrt{q})^2]$, $q = m/n$, as $m, n \to \infty$; anything outside that bulk is structure that noise of the same scale cannot produce. Heavy-tailed self-regularization [12] associates $\alpha \in [2, 4]$ with well-trained layers.

At initialization every network sits inside the bulk, which validates the measurement itself (Table 3, Figure 1). Training concentrates the convolutional weights: stable rank roughly halves, 7.6–15% of eigenvalues leave the bulk, and the tails reach the $\alpha \approx 3$–4.6 range. The small CNN acquires most of this structure in the first epoch, ResNet-18 and the ViT gradually. On EuroSAT the ViT moves least, in line with its lower accuracy, which foreshadows Section 6.

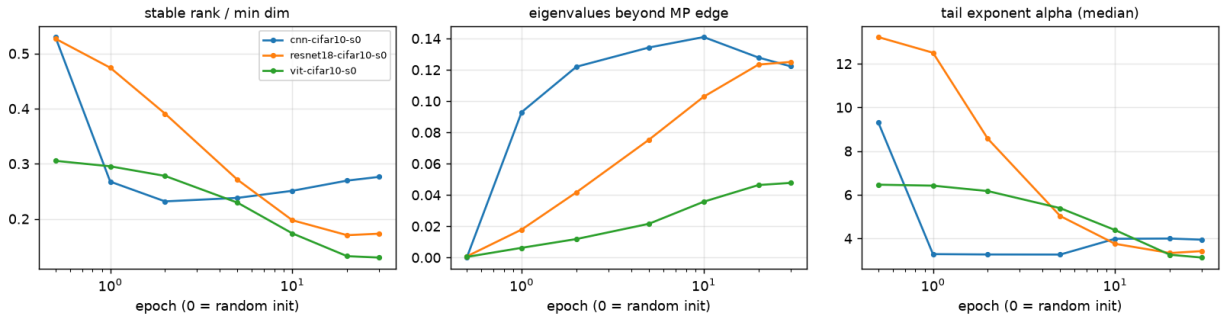


Figure 1: Stable-rank ratio, outlier fraction and tail exponent across training (CIFAR-10).

Run	$\lambda_{\max}$	Trace	Test accuracy
batch 32	44.5	187	**98.30%**
batch 128	72.0	402	98.26%
batch 256	100.7	669	97.78%
batch 512	240.7	980	97.11%
batch 128 + SAM	**40.5**	**118**	97.89%

Table 4: Curvature of the final (epoch 30) checkpoints, small CNN on EuroSAT.

5 Loss curvature

We measure the Hessian of the mean training loss on a fixed, class-balanced set of 512 un-augmented training images, with the network in evaluation mode (so batch normalization is a fixed affine map). Per checkpoint: the top three eigenvalues from 30 Lanczos steps, and the trace and density from SLQ with two probes of 40 steps, about 110 Hessian-vector products in total. The small CNN is retrained on EuroSAT with batch sizes 32, 128, 256 and 512 (everything else fixed) and once with SAM [2] ($\rho = 0.05$) at batch 128.

Larger batches end in sharper minima (Table 4): from batch 32 to 512 the top eigenvalue grows 5.4× and the trace 5.2×, monotonically, and test accuracy drops by 1.2 points, the picture of [7] reproduced at small scale. SAM does exactly what it optimizes for, cutting $\lambda_{\max}$ by 44% and the trace by 71% relative to plain batch 128, below even the batch-32 run, but it does not improve accuracy here (-0.37 points). On a dataset where every run is already at 97–98%, flatness and accuracy are correlated across batch sizes without SAM's extra flatness translating into accuracy; with one seed per run, differences below about 0.3 points are within noise. Curvature jumps from ≈ 0.5 at initialization to its peak within the first two epochs and relaxes as the learning rate decays (Figure 2).

The cost is dominated by the convolution double backward, for which cuDNN has no fast kernels: a single Hessian-vector product over 2048 images took 2.4 s on the laptop GPU, so the estimate uses 512 images (about 72 s per checkpoint, 43 minutes for all 35).

6 Low-rank compression

A linear layer $W \in \mathbb{R}^{m \times n}$ is replaced by its rank-r truncated SVD, i.e. two layers through an r-dimensional bottleneck, and a $k \times k$ convolution either by an SVD of its $c_{\text{out}} \times c_{\text{in}}k^2$ matrix (a

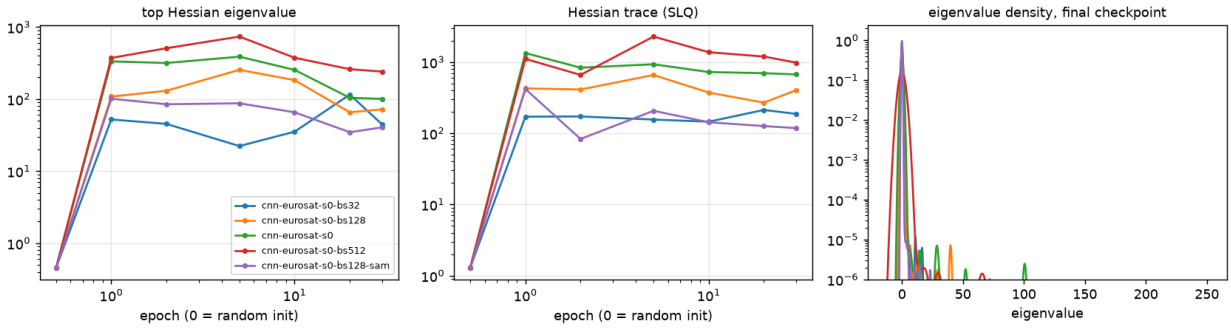


Figure 2: Top eigenvalue and trace across training (log scales) and the SLQ density of the final checkpoints.

Model (dense acc.)	Method, energy	MACs	Params	No fine-tune	2-epoch fine-tune
CNN (97.78%)	Tucker-2, 0.90	**45.1%**	49.5%	94.52%	**97.78%**
	Tucker-2, 0.80	22.6%	22.5%	71.26%	97.22%
	SVD, 0.80	40.6%	40.0%	70.74%	97.70%
ResNet-18 (98.00%)	Tucker-2, 0.80	**58.9%**	60.6%	95.37%	97.67%
	SVD, 0.80	72.8%	74.8%	95.81%	98.04%
	Tucker-2, 0.90	84.5%	85.3%	96.48%	98.00%
ViT (93.56%)	SVD, 0.80	88.7%	87.9%	92.85%	93.63%

Table 5: Compression of the final EuroSAT checkpoints. MACs and parameters relative to the dense model.

$k \times k$ convolution to r channels followed by a 1×1) or by a Tucker-2 decomposition [8]: HOSVD on the output- and input-channel modes gives

$$\mathcal{W} \approx \mathcal{G} \times_1 U_{\text{out}} \times_2 U_{\text{in}}, \qquad \mathcal{G} \in \mathbb{R}^{r_{\text{out}} \times r_{\text{in}} \times k \times k},$$

implemented as 1×1 ($c_{\text{in}} \to r_{\text{in}}$), $k \times k$ core ($r_{\text{in}} \to r_{\text{out}}$), 1×1 ($r_{\text{out}} \to c_{\text{out}}$). Ranks keep a fraction ("energy") of the squared singular values of each unfolding, and a layer is replaced only if the factorization has fewer parameters. Costs are counted in multiply-accumulates (MACs) from one forward pass, including attention's $QK^\top$ and AV products. After compression the network is evaluated as is and after two epochs of fine-tuning (SGD, learning rate 0.01, cosine).

Tucker-2 dominates SVD on convolutions at every budget, because it compresses both channel modes instead of folding the spatial kernel into one side (Table 5, Figure 3). Two epochs of fine-tuning recover almost everything: the CNN keeps its accuracy at 45% of the MACs and ResNet-18 loses 0.33 points at 59%. Fine-tuning the dense models the same way changes them by at most +0.1 points, so this is recovery rather than additional training. The ViT barely compresses: at 95% energy no factorization is smaller than the dense layer, consistent with its weight spectra moving least in Section 4.

Does stable rank predict sensitivity? Each eligible layer is truncated alone to 25% of its rank and the accuracy drop (no fine-tuning) is compared with the layer's stable-rank ratio. The Spearman correlation is +0.70 over the CNN's 5 layers, +0.31 over ResNet-18's 19 and −0.22 over the ViT's 12 (Figure 4). The sign is the expected one on the CNNs, but with $\rho \approx 0.3$ on ResNet-18

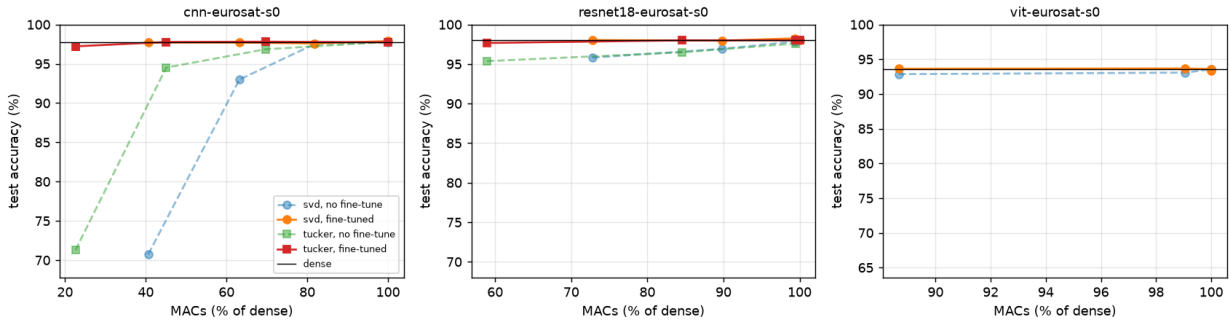


Figure 3: Test accuracy against MACs, before (dashed) and after (solid) fine-tuning.

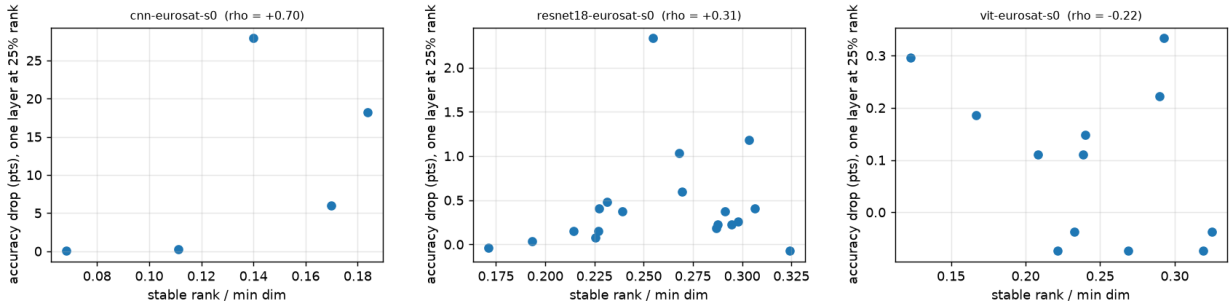


Figure 4: Accuracy drop when one layer is truncated to 25% of its rank, against its stable-rank ratio.

the spectrum alone is not a rule one could choose ranks by; where a layer sits in the network matters at least as much.

7 Limitations

Every configuration was trained with a single seed, so differences of a few tenths of a point are not significant. The Hessian is estimated on 512 images in evaluation mode, which is not the function SGD actually optimizes with batch normalization in training mode. The compression study uses an energy threshold shared by all layers; per-layer rank selection (e.g. by measured sensitivity or a FLOP budget) would likely do better than any of the uniform settings reported. The models are small by modern standards, chosen so that every spectrum could be computed exactly; whether the same trends hold for large pretrained networks is not tested here.

8 Conclusion

Training turns random-looking weight matrices into low stable-rank, heavy-tailed ones within the first few epochs; larger batches end in sharper minima that generalize slightly worse, and SAM flattens the minimum without, on this dataset, improving accuracy; and the low-rank structure that training creates in convolutional layers can be cashed in as a 41–55% reduction in compute at little or no accuracy cost after a short fine-tune, while spectra by themselves are only a weak guide to which layers to compress.

A Reproducing the results

```
python -m spectralens data eurosat --download
python -m spectralens.train model=cnn,resnet18,vit \
    data=eurosat,cifar10
python -m spectralens.spectra
# batch-size sweep (also 128, 512) and SAM
python -m spectralens.train model=cnn data=eurosat \
    batch_size=32 tag=bs32
python -m spectralens.train model=cnn data=eurosat \
    batch_size=128 sam_rho=0.05 tag=bs128-sam
python -m spectralens.hessian --name hessian_batch_size \
    --run cnn-eurosat-s0-bs32 --run cnn-eurosat-s0-bs128 ...
python -m spectralens.compress.run
```

References

[1] Carl Eckart and Gale Young. The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3):211–218, 1936.

[2] Pierre Foret, Ariel Kleiner, Hossein Mobahi, and Behnam Neyshabur. Sharpness-aware minimization for efficiently improving generalization. In *International Conference on Learning Representations (ICLR)*, 2021.

[3] Behrooz Ghorbani, Shankar Krishnan, and Ying Xiao. An investigation into neural net optimization via hessian eigenvalue density. In *International Conference on Machine Learning (ICML)*, 2019.

[4] Nathan Halko, Per-Gunnar Martinsson, and Joel A. Tropp. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288, 2011.

[5] Patrick Helber, Benjamin Bischke, Andreas Dengel, and Damian Borth. Eurosat: A novel dataset and deep learning benchmark for land use and land cover classification. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 12(7):2217–2226, 2019.

[6] Michael F. Hutchinson. A stochastic estimator of the trace of the influence matrix for Laplacian smoothing splines. *Communications in Statistics - Simulation and Computation*, 18(3):1059–1076, 1989.

[7] Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang. On large-batch training for deep learning: Generalization gap and sharp minima. In *International Conference on Learning Representations (ICLR)*, 2017.

[8] Yong-Deok Kim, Eunhyeok Park, Sungjoo Yoo, Taelim Choi, Lu Yang, and Dongjun Shin. Compression of deep convolutional neural networks for fast and low power mobile applications. In *International Conference on Learning Representations (ICLR)*, 2016.

[9] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.

[10] Lin Lin, Yousef Saad, and Chao Yang. Approximating spectral densities of large matrices. *SIAM Review*, 58(1):34–65, 2016.

[11] Vladimir A. Marchenko and Leonid A. Pastur. Distribution of eigenvalues for some sets of random matrices. *Mathematics of the USSR-Sbornik*, 1(4):457–483, 1967.

[12] Charles H. Martin and Michael W. Mahoney. Implicit self-regularization in deep neural networks: Evidence from random matrix theory and implications for learning. *Journal of Machine Learning Research*, 22(165):1–73, 2021.

[13] Raphael A. Meyer, Cameron Musco, Christopher Musco, and David P. Woodruff. Hutch++: Optimal stochastic trace estimation. In *Symposium on Simplicity in Algorithms (SOSA)*, 2021.

[14] Barak A. Pearlmutter. Fast exact multiplication by the hessian. *Neural Computation*, 6(1): 147–160, 1994.

[15] Yousef Saad. *Numerical Methods for Large Eigenvalue Problems*. SIAM, 2nd edition, 2011.